

Angular Dependency Injection

Class-based vs InjectionToken

 **Master DI Patterns**

Write Flexible & Testable Code



The Problem



Class-based DI Limitations

- X Tight Coupling:** Components directly depend on concrete classes
- X Hard to Replace:** Swapping implementations requires code changes
- X Testing Pain:** Difficult to mock dependencies
- X No Flexibility:** Can't switch strategies easily



Class-based DI

```
// logger.service.ts

@Injectable({ providedIn: 'root' })
export class LoggerService {
  log(msg: string) {
    console.log(msg);
  }
}
```

```
// app.component.ts

constructor(
  private logger: LoggerService
) {}
```

✗ Tightly Coupled

1

Define Interface

```
// logger.interface.ts

export interface ILogger {
  log(message: string): void;
}
```

✨ Create a contract that defines behavior

Implement Interface

```
// console-logger.service.ts

export class ConsoleLoggerService
  implements ILogger {

  log(message: string): void {
    console.log('Console:', message);
  }
}
```

💡 **Concrete implementation of the contract**

3

Create InjectionToken

```
// token.ts
```

```
import { InjectionToken } from '@angular/core';
```

```
export const LOGGER_TOKEN =  
  new InjectionToken<ILogger>(  
    'LOGGER_TOKEN'  
  );
```



Token to identify the dependency

Configure & Inject

```
// app.module.ts - Provider
```

```
providers: [  
  {  
    provide: LOGGER_TOKEN,  
    useClass: ConsoleLoggerService  
  }  
]
```

```
// app.component.ts - Injection
```

```
constructor(  
  @Inject(LOGGER_TOKEN)  
  private logger: ILogger  
) {}
```



Loosely Coupled



The Difference

Class-based DI

- ✗ Tight coupling
- ✗ Hard to swap
- ✗ Testing difficult
- ✗ Code changes needed
- ✗ Less flexible

InjectionToken DI

- ✓ Loose coupling
- ✓ Easy to swap
- ✓ Testing simple
- ✓ Config changes only
- ✓ Highly flexible



Key Benefits



Easy Testing

Inject mock implementations without changing component code



Swap Anytime

Change implementations by updating provider config only



Environment-Based

Different implementations for dev, staging, and production



Multiple Implementations

Console, File, Remote loggers - switch effortlessly

Write Better Angular Code Today! 🦖

Use InjectionToken for flexible,
testable, and maintainable applications

Remember:

Depend on abstractions,
not concrete implementations